

¡Entendido perfectamente! Disculpa por haber resumido demasiado anteriormente. He captado el mensaje: quieres una **GUÍA MAESTRA Y EXHAUSTIVA**. Un documento de referencia completo donde esté *todo* lo que aparece en tus materiales, desde la historia hasta el último detalle técnico, sin atajos, y con ejemplos prácticos enteros.

Prepárate, porque este es el manual definitivo basado en toda la documentación que has aportado.

GUÍA MAESTRA DE LENGUAJES DE MARCAS Y DESARROLLO WEB

Esta guía recopila toda la teoría, historia, reglas de sintaxis y prácticas de los documentos proporcionados sobre XML, DTD, JSON/APIs y HTML/CSS.

MÓDULO 1: XML (eXtensible Markup Language)

1.1. Teoría e Historia Profunda

El Problema Original (SGML y HTML):

Antes de XML, existía SGML (Standard Generalized Markup Language), un estándar internacional muy potente para definir lenguajes de marcado, pero extremadamente complejo y difícil de implementar en la web incipiente. Por otro lado, estaba HTML, que era simple pero tenía un gran problema: mezclaba la estructura de los datos con su presentación (cómo se ven). Además, HTML tenía un conjunto de etiquetas fijo y limitado (<p>, <table>, etc.).

El Nacimiento de XML:

A finales de los años 90, el W3C (World Wide Web Consortium) buscó una solución intermedia. Necesitaban la potencia y flexibilidad de SGML para definir estructuras de datos personalizadas, pero adaptada a la simplicidad que requería la Web.

El resultado fue XML (eXtensible Markup Language). Se define como un **metalenguaje**: un lenguaje para crear otros lenguajes.

Filosofía Clave:

1. **Separación de Preocupaciones:** XML se diseñó **exclusivamente para estructurar, almacenar y transportar datos**. NO para mostrarlos. XML no sabe cómo se ven los datos; eso se deja para CSS o XSLT.
 2. **Extensibilidad (La 'X'):** No hay etiquetas predefinidas. Tú defines tu propio vocabulario. Si tu negocio es una biblioteca, creas etiquetas <libro> y <autor>. Si es un taller, creas <coche> y <matricula>.
 3. **Legibilidad:** Debe ser legible tanto por humanos como por máquinas.
-

1.2. Sintaxis y Reglas: El Documento "Bien Formado"

Para que un procesador XML acepte un archivo, este debe estar "Bien Formado" (Well-Formed). Esto significa cumplir estrictamente las reglas sintácticas de XML. XML no perdona errores de sintaxis como HTML.

A. El Prólogo (Declaración XML)

Es la primera línea del archivo. No es obligatoria, pero sí altamente recomendada.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

- version="1.0": Indica la versión de XML utilizada.
- encoding="UTF-8": Vital para idiomas como el español. Asegura que caracteres como tildes (á, é) o la eñe (ñ) se interpreten correctamente y no aparezcan símbolos extraños.

B. Regla del Elemento Raíz Único

Todo documento XML debe tener **exactamente un** elemento principal que englobe a todos

los demás elementos. Es el contenedor padre de todo el documento.

Incorrecto (Múltiples raíces):

XML

```
<coche>Honda</coche>  
<coche>Ford</coche>
```

Correcto (Raíz única):

XML

```
<concesionario>  
  <coche>Honda</coche>  
  <coche>Ford</coche>  
</concesionario>
```

C. Reglas de las Etiquetas (Tags)

1. **Cierre Obligatorio:** Toda etiqueta que se abre <etiqueta> debe cerrarse </etiqueta>.
2. **Elementos Vacíos:** Si un elemento no tiene contenido, se puede usar la sintaxis abreviada de auto-cierre.
 - Forma larga: <foto></foto>
 - Forma abreviada: <foto/>
3. **Sensibilidad a Mayúsculas (Case Sensitive):** XML distingue entre mayúsculas y minúsculas. <Libro> es diferente de <libro>. Se recomienda usar una convención constante (generalmente minúsculas o camelCase).
4. **Nombres de Etiquetas:** Deben empezar por una letra o guion bajo _. Pueden contener letras, números, guiones -, puntos . y guiones bajos. No pueden contener espacios ni empezar por xml.

D. Regla de Anidamiento Correcto

Los elementos deben cerrarse en el orden inverso al que se abrieron. No pueden solaparse "cruzándose".

Incorrecto (Solapado):

XML

```
<titulo>Historia de <i>España</titulo></i>
```

Correcto (Anidado):

XML

```
<titulo>Historia de <i>España</i></titulo>
```

E. Reglas de los Atributos

Los atributos proporcionan metadatos o información adicional sobre un elemento y van en la etiqueta de apertura.

1. **Comillas Obligatorias:** Los valores de los atributos **siempre** deben estar encerrados entre comillas (simples ' o dobles ").
 - Incorrecto: <libro id=123>
 - Correcto: <libro id="123">
2. **Unicidad:** Un atributo no puede repetirse en la misma etiqueta. <coche color="rojo" color="azul"> es inválido.

F. Caracteres Especiales (Entidades Predefinidas)

Ciertos caracteres tienen un significado especial en XML (como < y >) y no pueden usarse directamente dentro del texto de un elemento, ya que confundirían al procesador. Deben sustituirse por "entidades":

Carácter	Nombre	Entidad a usar
<	Menor que	< (less than)
>	Mayor que	> (greater than)
&	Ampersand	&
"	Comillas dobles	"
'	Apóstrofe	'

Ejemplo: Para escribir "A < B", en XML se escribe: <ecuacion>A < B</ecuacion>.

G. Secciones CDATA (Character Data)

A veces necesitas incluir un bloque de texto grande que contiene muchos caracteres especiales (como código de programación) y no quieres escapar cada uno con entidades. Puedes usar una sección CDATA. Todo lo que esté dentro se interpreta literalmente como texto puro y el procesador XML no intenta analizar las etiquetas dentro.

XML

```
<codigo_ejemplo>
<![CDATA[
  function comparar(a, b) {
    if (a < b && a > 0) {
      return true;
    }
  }
]
```

```
}  
]]>  
</codigo_ejemplo>
```

1.3. Ejemplo Práctico Completo: Fichero XML

Este es un ejemplo completo y "Bien Formado" basado en tus ejercicios de coches, aplicando todas las reglas anteriores.

Archivo: inventario_coches.xml

XML

```
<?xml version="1.0" encoding="UTF-8"?>  
<oferta_vehiculos>  
  <vehiculo id="v001" tipo="turismo">  
    <marca>Ford</marca>  
    <modelo color="Blanco Nieve">Focus RS</modelo>  
    <motor combustible="Gasolina">2.3 EcoBoost 350CV</motor>  
    <matricula>1234 BBC</matricula>  
    <descripcion>Vehículo de "alto rendimiento". Ideal para circuito & carretera.</descripcion>  
    <kilometros>45000</kilometros>  
    <precio_inicial moneda="EUR">35000.00</precio_inicial>  
    <precio_oferta moneda="EUR">32500.00</precio_oferta>  
    <extra valor="500">Pintura metalizada</extra>  
    <extra valor="1200">Paquete Performance</extra>  
    <foto url="imagenes/ford_focus_rs.jpg"/>  
  </vehiculo>  
  
  <vehiculo id="v002" tipo="suv">  
    <marca>Toyota</marca>  
    <modelo color="Gris Grafito">RAV4 Híbrido</modelo>  
    <motor combustible="Híbrido">2.5 Dynamic Force 222CV</motor>  
    <matricula>5678 JKL</matricula>  
    <descripcion>SUV eficiente y espacioso.</descripcion>  
    <kilometros>15000</kilometros>
```

```
<precio_inicial moneda="EUR">38000.00</precio_inicial>
<foto url="imagenes/toyota_rav4.jpg"/>
</vehiculo>
</oferta_vehiculos>
```

MÓDULO 2: DTD (Document Type Definition)

2.1. Teoría y Propósito

Bien Formado vs. Válido:

Un XML "bien formado" solo significa que cumple las reglas sintácticas básicas (etiquetas cerradas, comillas en atributos, etc.). Pero eso no garantiza que los datos tengan sentido para tu negocio. Por ejemplo, un XML podría estar "bien formado" pero tener un <precio> que contiene texto en lugar de números, o faltarle la <matricula> a un coche.

Un **DTD (Definición de Tipo de Documento)** es un archivo de reglas que define la **estructura válida** de un XML específico. Es un "contrato" que el XML debe cumplir.

Si un XML cumple las reglas de su DTD, se dice que es un documento "**Válido**".

¿Qué define un DTD?

1. Los elementos permitidos.
2. El orden exacto de los elementos hijos.
3. La cantidad de veces que pueden aparecer (cardinalidad).
4. Qué elementos pueden contener texto y cuáles deben estar vacíos.
5. Los atributos permitidos para cada elemento, sus tipos y si son obligatorios u opcionales.

Tipos de DTD:

- **Interno:** Las reglas se escriben dentro del mismo archivo XML, en el prólogo. Útil para pruebas rápidas.
 - **Externo:** Las reglas están en un archivo separado (.dtd). Es lo más profesional, permite que muchos XML compartan las mismas reglas.
-

2.2. Sintaxis y Reglas Exhaustivas del DTD

El DTD tiene su propia sintaxis, diferente a la de XML. Se basa en declaraciones que comienzan con <!.

A. Declaración de Elementos (<!ELEMENT ...>)

Define un elemento y su contenido permitido.

Sintaxis: <!ELEMENT nombre_elemento (modelo_de_contenido)>

Tipos de Modelo de Contenido:

1. Texto Analizable (#PCDATA):

Significa "Parsed Character Data". El elemento solo puede contener texto, sin etiquetas hijas.

Fragmento de código

```
<!ELEMENT matricula (#PCDATA)>
```

2. Elemento Vacío (EMPTY):

El elemento no puede tener contenido de texto ni elementos hijos.

Fragmento de código

```
<!ELEMENT salto_linea EMPTY>
```

3. Cualquier Contenido (ANY):

Permite cualquier combinación de texto y elementos declarados en el DTD. Es muy permisivo y se usa poco.

Fragmento de código

```
<!ELEMENT notas ANY>
```

4. Contenido de Elementos Hijos (Estructura):

Define qué elementos hijos debe contener, en qué orden y cuántos. Se usan paréntesis y operadores.

- **Operadores de Secuencia y Alternativa:**

- Coma (,): Secuencia. Los elementos deben aparecer en ese orden exacto.

```
<!ELEMENT persona (nombre, apellido)> ->
```

```
<persona><nombre>N</nombre><apellido>A</apellido></persona>
```

- Barra Vertical (|): Alternativa. Debe aparecer uno U otro elemento de la lista.

```
<!ELEMENT contacto (email | telefono)> -> Permite <email> O <telefono>, pero no ambos a la vez ni ninguno.
```

- Operadores de Cardinalidad (Ocurrecia):

Se colocan justo después del nombre del elemento o del grupo entre paréntesis.

- **(Ningún símbolo):** Debe aparecer **exactamente una vez**.

- **? (Interrogación):** Opcional. Aparece **cero o una vez**.
- **+ (Más):** Obligatorio repetible. Aparece **una o más veces**.
- *** (Asterisco):** Opcional repetible. Aparece **cero o más veces**.
- **Ejemplos Complejos de Combinación:**

Fragmento de código

```
<!ELEMENT coche (marca, modelo, descripcion?, foto+)>
```



```
<!ELEMENT capitulo (titulo, (parrafo | imagen)* )>
```

B. Declaración de Atributos (<!ATTLIST ...>)

Define los atributos permitidos para un elemento específico.

Sintaxis: <!ATTLIST nombre_elemento nombre_atributo tipo_atributo modificador_uso ... >

Puedes definir varios atributos para un mismo elemento uno debajo del otro.

1. Tipos de Atributo:

- **CDATA (Character Data):** El tipo más común. Admite cualquier cadena de texto.
<!ATTLIST modelo color CDATA ...>
- **ID:** Un identificador único para ese elemento en todo el documento XML.
 - Debe empezar por letra o guion bajo.
 - No puede haber dos elementos en el XML con el mismo valor de ID.
 - Un elemento solo puede tener un atributo de tipo ID.
 <!ATTLIST vehiculo codigo_bastidor ID ...>
- **IDREF (ID Reference):** El valor de este atributo debe ser el mismo que un valor de un ID que ya exista en otro elemento del documento. Sirve para crear relaciones o enlaces internos.
<!ATTLIST conductor coche_asignado IDREF ...>
- **IDREFS:** Igual que IDREF, pero permite una lista de referencias separadas por espacios.
- **Enumeración (valor1 | valor2 | ...):** Restringe el valor del atributo a una lista cerrada de opciones.
<!ATTLIST motor combustible (Gasolina | Diesel | Híbrido | Eléctrico) ...>
- **NMTOKEN / NMTOKENS:** Similar a CDATA pero más restrictivo. Solo permite letras, números, puntos, guiones y guiones bajos. No permite espacios.

2. Modificadores de Uso (Reglas):

- **#REQUIRED:** El atributo es obligatorio. Si falta en el XML, el documento no es válido.
<!ATTLIST foto url CDATA #REQUIRED>
- **#IMPLIED:** El atributo es opcional.
<!ATTLIST vehiculo fecha_entrada CDATA #IMPLIED>

- **#FIXED "valor"**: El atributo es opcional, pero si aparece, DEBE tener ese valor exacto. Si no aparece, el procesador asume que tiene ese valor.
<!ATTLIST oferta version_api CDATA #FIXED "2.0">
- **"valor_por_defecto"**: Si el atributo no se escribe en el XML, el procesador asume este valor.
<!ATTLIST coche estado (nuevo | usado) "nuevo"> (Si no se especifica estado, se asume "nuevo").

C. Declaración de Entidades (<!ENTITY ...>)

Son como variables o macros que se definen en el DTD y se usan en el XML.

- **Entidades Internas**: Definen una cadena de sustitución de texto.
 - En DTD: <!ENTITY nombre_empresa "Automóviles y Talleres Manolo S.L.">
 - En XML: <footer texto="Derechos reservados por &nombre_empresa;"/>
- **Entidades Externas**: Permiten incluir el contenido de un archivo externo dentro del XML.
 - En DTD: <!ENTITY aviso_legal SYSTEM "textos/legal.txt">
 - En XML: <seccion_legal>&aviso_legal;</seccion_legal>

2.3. Ejemplo Práctico Completo: DTD y Vinculación

Vamos a crear el DTD externo para validar el XML de inventario_coches.xml que creamos en el Módulo 1.

Archivo: concesionario.dtd

Fragmento de código

```
<!ELEMENT oferta_vehiculos (vehiculo+)>
```

```
<!ELEMENT vehiculo (marca, modelo, motor, matricula, descripcion?, kilometros, precio_inicial, precio_oferta?, extra*, foto+)>
```

```
<!ATTLIST vehiculo
  id ID #REQUIRED
```

```
tipo (turismo | suv | deportivo | furgoneta) #REQUIRED>
```

```
<!ELEMENT marca (#PCDATA)>
```

```
<!ELEMENT matricula (#PCDATA)>
```

```
<!ELEMENT descripcion (#PCDATA)>
```

```
<!ELEMENT kilometros (#PCDATA)>
```

```
<!ELEMENT modelo (#PCDATA)>
```

```
<!ATTLIST modelo color CDATA #REQUIRED>
```

```
<!ELEMENT motor (#PCDATA)>
```

```
<!ATTLIST motor combustible (Gasolina | Diesel | Híbrido | Eléctrico) #REQUIRED>
```

```
<!ELEMENT precio_inicial (#PCDATA)>
```

```
<!ATTLIST precio_inicial moneda CDATA "EUR">
```

```
<!ELEMENT precio_oferta (#PCDATA)>
```

```
<!ATTLIST precio_oferta moneda CDATA "EUR">
```

```
<!ELEMENT extra (#PCDATA)>
```

```
<!ATTLIST extra valor CDATA #REQUIRED>
```

```
<!ELEMENT foto EMPTY>
```

```
<!ATTLIST foto url CDATA #REQUIRED>
```

Cómo vincular este DTD en el archivo XML:

Se añade la declaración <!DOCTYPE> justo después del prólogo XML.

Archivo: inventario_coches.xml (con vinculación DTD)

XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE oferta_vehiculos SYSTEM "concesionario.dtd">
```

```
<oferta_vehiculos>
```

```
  <vehiculo id="v001" tipo="turismo">
```

```
    </vehiculo>
```

```
</oferta_vehiculos>
```

MÓDULO 3: JSON y APIs REST

3.1. Teoría e Historia: El Sucesor de XML en la Web

El Auge de la Web Dinámica (AJAX):

A mediados de los 2000, la web empezó a cambiar. Las páginas dejaron de ser documentos estáticos para convertirse en aplicaciones dinámicas que recargaban datos en segundo plano sin refrescar la página completa (tecnología AJAX).

En ese entorno, XML resultaba pesado. Es muy "verboso" (muchas etiquetas de apertura y cierre que ocupan ancho de banda) y, sobre todo, es difícil de procesar (parsear) dentro de un navegador web usando JavaScript.

JSON (JavaScript Object Notation):

JSON nació como la solución natural. Fue popularizado por Douglas Crockford. No es un lenguaje nuevo, sino que es literalmente un subconjunto de la sintaxis que usa el lenguaje de programación JavaScript para definir objetos.

Ventajas de JSON sobre XML en la Web:

1. **Ligereza:** Es menos verboso, los archivos son más pequeños y se transmiten más rápido.
2. **Nativo en JavaScript:** Los navegadores pueden convertir un texto JSON en un objeto JavaScript usable con una sola línea de código (`JSON.parse()`), sin necesidad de parsers complejos.
3. **Tipos de Datos:** XML solo entiende de texto. JSON entiende nativamente de números, booleanos, arrays y objetos, lo que facilita el trabajo a los programadores.

APIs REST (Representational State Transfer):

Es el estilo arquitectónico dominante hoy en día para diseñar servicios web (APIs) que permiten la comunicación entre diferentes aplicaciones (por ejemplo, una app móvil hablando con un servidor, o tu navegador pidiendo datos a la PokeAPI).

Se basa en usar los estándares existentes de la web, principalmente el protocolo HTTP, de forma inteligente.

Principios básicos de REST:

1. **Recursos:** Todo es un recurso (un coche, un usuario, una factura) y se identifica con una URL única (ej. `https://api.concesionario.com/coches/v001`).
2. **Verbos HTTP:** Se usan los métodos del protocolo HTTP para indicar qué acción queremos hacer sobre ese recurso (Leer, Crear, Actualizar, Borrar).
3. **Sin Estado (Stateless):** Cada petición del cliente al servidor debe ser independiente y contener toda la información necesaria (incluyendo autenticación). El servidor no

recuerda la "sesión" anterior del cliente.

4. **Representaciones:** Los recursos se intercambian en formatos estándar, siendo JSON el más utilizado actualmente.

3.2. Sintaxis y Reglas Exhaustivas de JSON

JSON es muy estricto con su sintaxis. Un solo error (como una coma de más) invalida todo el archivo.

1. Estructuras Principales:

- **Objetos {}:** Colecciones no ordenadas de pares clave:valor. Se delimitan por llaves.

```
JSON
{
  "clave1": "valor1",
  "clave2": 123
}
```

- **Arrays []:** Listas ordenadas de valores. Se delimitan por corchetes.

```
JSON
[ "valor1", "valor2", "valor3" ]
```

2. Reglas de Sintaxis Estrictas:

- **Comillas en Claves:** TODAS las claves (nombres de propiedad) en un objeto DEBEN ir entre comillas dobles.
 - Incorrecto: { nombre: "Juan" }
 - Correcto: { "nombre": "Juan" }
- **Comillas en Cadenas:** Los valores de tipo texto (string) DEBEN ir entre comillas dobles. Las comillas simples ' NO son válidas en JSON estándar.
- **Separadores:**
 - Los dos puntos : separan la clave del valor.
 - La coma , separa los pares dentro de un objeto o los elementos dentro de un array.
- **Trailing Comma (Coma final):** NO debe haber una coma después del último elemento de un objeto o array. Esto es un error muy común.
 - Incorrecto: [1, 2, 3,]
 - Correcto: [1, 2, 3]

3. Tipos de Datos Permitidos en los Valores:

- **Cadena (String):** Texto entre comillas dobles. "Hola Mundo". Caracteres especiales

como las comillas dobles o la barra invertida dentro del texto deben escaparse con \: "Dijo: \"Hola\"".

- **Número (Number):** Enteros o decimales. No llevan comillas. 100, -5.5, 1.2e3 (notación científica).
- **Booleano (Boolean):** true o false. Siempre en minúsculas y sin comillas.
- **Nulo (Null):** null. Indica la ausencia de valor. Siempre en minúsculas y sin comillas.
- **Objeto:** Un objeto JSON anidado dentro de otro. { "autor": { "nombre": "...", "edad": ... } }
- **Array:** Un array JSON anidado. { "telefonos": ["111", "222"] }

4. Transformación XML a JSON (Ejemplo Práctico):

Observa cómo la estructura XML se traduce a JSON, especialmente cómo los elementos repetidos (<extra>) se convierten en un array de objetos.

XML Original:

XML

```
<vehiculo id="v001" tipo="turismo">
  <marca>Ford</marca>
  <modelo color="Blanco">Focus RS</modelo>
  <kilometros>45000</kilometros>
  <activo>true</activo>
  <extra valor="500">Pintura</extra>
  <extra valor="1200">Pack Performance</extra>
</vehiculo>
```

JSON Equivalente:

JSON

```
{
  "id": "v001",
  "tipo": "turismo",
  "marca": "Ford",
  "modelo": {
    "nombre": "Focus RS",
```

```
{
  "color": "Blanco",
},
"kilometros": 45000,
"activo": true,
"extras": [
  { "nombre": "Pintura", "valor": 500 },
  { "nombre": "Pack Performance", "valor": 1200 }
]
}
```

3.3. Guía Práctica Completa: Uso de APIs REST con HTTP (Postman)

Las APIs REST utilizan los verbos del protocolo HTTP para definir qué operación (CRUD) realizar sobre los datos.

A. Los Verbos HTTP Principales (CRUD)

Verbo HTTP	Acción CRUD	Descripción	Ejemplo de Uso	Cuerpo en Petición?
GET	Read (Leer)	Solicita datos de un recurso. No debe modificar nada en el servidor.	Obtener lista de coches o detalles de uno.	No
POST	Create (Crear)	Envía datos para crear un nuevo recurso en el servidor.	Añadir un nuevo coche al inventario.	Sí (Datos del nuevo coche en JSON)
PUT	Update (Reemplazar)	Actualiza un recurso existente	Modificar todos los datos de un	Sí (Datos completos del coche en

		reemplazándolo o completamente con los nuevos datos enviados.	coche existente.	JSON)
PATCH	Update (Modificar)	Actualiza parcialmente un recurso. Solo se envían los campos que cambian.	Cambiar solo el precio y los kilómetros de un coche.	Sí (JSON parcial con los cambios)
DELETE	Delete (Borrar)	Solicita la eliminación de un recurso.	Borrar un coche del inventario.	Generalmente No

B. Códigos de Estado HTTP (Status Codes)

El servidor responde con un número de 3 dígitos que indica el resultado de la operación. Es vital entenderlos.

- **2xx: Éxito (Success)**
 - **200 OK:** La petición se realizó correctamente. Es la respuesta estándar para GET, PUT, DELETE.
 - **201 Created:** Específico para POST. Indica que el recurso se ha creado con éxito en el servidor.
 - **204 No Content:** Éxito, pero la respuesta no tiene cuerpo (común en DELETE o PUT).
- **4xx: Error del Cliente (Client Error) - Es tu culpa**
 - **400 Bad Request:** La petición está mal formada (ej. JSON inválido, faltan datos obligatorios).
 - **401 Unauthorized:** Se requiere autenticación y no se ha proporcionado o ha fallado.
 - **403 Forbidden:** El cliente está autenticado pero no tiene permisos para realizar esa acción.
 - **404 Not Found:** El recurso solicitado no existe en el servidor (la URL está mal o el ID no existe).
- **5xx: Error del Servidor (Server Error) - Es culpa del servidor**

- **500 Internal Server Error:** Error genérico e inesperado en el servidor (bug, base de datos caída, etc.).
- **503 Service Unavailable:** El servidor no está disponible momentáneamente (sobrecarga, mantenimiento).

C. Ejemplo Completo de Interacción API (Simulación en Postman)

Imaginemos que la API de nuestro concesionario está en <https://api.concesionario.com/v1>.

Escenario 1: Obtener el inventario (GET)

- **Método:** GET
- **URL:** <https://api.concesionario.com/v1/coches>
- **Respuesta (Status 200 OK):**

JSON

```
[  
  { "id": "v001", "marca": "Ford", "modelo": "Focus RS", "precio": 32500 },  
  { "id": "v002", "marca": "Toyota", "modelo": "RAV4", "precio": 38000 }  
]
```

Escenario 2: Crear un coche nuevo (POST)

- **Método:** POST
- **URL:** <https://api.concesionario.com/v1/coches>
- **Headers:** Content-Type: application/json (Obligatorio para avisar que enviamos JSON).
- **Body (raw JSON):**

JSON

```
{  
  "marca": "Seat",  
  "modelo": "León Cupra",  
  "motor": "2.0 TSI 300CV",  
  "precio": 29000,  
  "extras": ["Techo solar"]  
}
```

- **Respuesta (Status 201 Created):**
El servidor suele devolver el objeto creado, incluyendo datos generados por él mismo (como el nuevo ID o fecha de creación).

JSON

```
{
```

```
{
  "id": "v003_generado_por_servidor",
  "marca": "Seat",
  "modelo": "León Cupra",
  "motor": "2.0 TSI 300CV",
  "precio": 29000,
  "extras": ["Techo solar"],
  "fecha_creacion": "2023-10-27T10:00:00Z"
}
```

MÓDULO 4: HTML5 y CSS3

4.1. Teoría e Historia: La Evolución de la Web

HTML (HyperText Markup Language): La Estructura

Creado por Tim Berners-Lee en 1991. Su propósito original era definir la estructura de documentos científicos para que pudieran ser compartidos y enlazados (hipertexto) a través de internet. No estaba pensado para el diseño gráfico.

El Problema de la Presentación:

En los años 90, con la explosión de la web comercial, los diseñadores querían controlar colores, fuentes y maquetación. Los navegadores empezaron a añadir etiquetas HTML no estándar para esto (como , <center>, <bgcolor>), convirtiendo el HTML en una "sopa de etiquetas" difícil de mantener, donde estructura y diseño estaban mezclados.

CSS (Cascading Style Sheets) al Rescate:

El W3C propuso CSS para solucionar esto. La idea fundamental es la Separación de

Preocupaciones:

- **HTML** se encarga SOLO de la **estructura y semántica** (qué es cada parte: esto es un título, esto es un párrafo, esto es una lista).
- **CSS** se encarga SOLO de la **presentación visual** (cómo se ve: colores, tamaños, márgenes, posición).

HTML5: La Web Semántica y Multimedia

HTML5 es la versión actual. No es solo una limpieza del lenguaje, sino una revolución que convierte a la web en una plataforma de aplicaciones. Sus pilares son:

1. **Semántica:** Nuevas etiquetas con significado real (<header>, <nav>, <article>, <section>, <footer>) para reemplazar el uso genérico de <div>. Esto es crucial para la accesibilidad (lectores de pantalla para ciegos) y el SEO (motores de búsqueda).

2. **Multimedia Nativo:** Etiquetas <audio> y <video> para reproducir medios sin depender de plugins externos problemáticos como Flash.
 3. **Formularios Potentes:** Nuevos tipos de entrada (date, email, number, range) que los navegadores validan automáticamente.
-

4.2. Sintaxis y Reglas Exhaustivas de HTML5 y CSS3

A. HTML5: Estructura y Semántica

1. Estructura Base Obligatoria:

HTML

```
<!DOCTYPE html> <html lang="es"> <head>
  <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Título de la pestaña del navegador</title>
  <link rel="stylesheet" href="estilos.css"> </head>
<body>
  </body>
</html>
```

2. Etiquetas Semánticas Principales (El fin de la "Sopa de DIVs"):

- **<header>**: Cabecera de la página o de una sección. Suele contener el logotipo, título principal o navegación.
- **<nav>**: Bloque de enlaces de navegación principales.
- **<main>**: Contenido principal y único de la página. Solo debe haber uno por página.
- **<section>**: Agrupación temática de contenido, generalmente con un encabezado.
- **<article>**: Contenido independiente y autónomo que tiene sentido por sí mismo (un post de blog, una noticia, una ficha de producto).
- **<aside>**: Contenido relacionado tangencialmente con el contenido principal (barras laterales, publicidad).
- **<footer>**: Pie de página o de una sección. Contiene copyright, enlaces de contacto, etc.

3. Elementos Clave:

- **Encabezados:** <h1> (más importante) a <h6> (menos importante). Mantener una jerarquía lógica.
- **Párrafos:** <p> Texto </p>.
- **Listas:**
 - Desordenadas (puntos): Elemento 1 Elemento 2
 - Ordenadas (números): Primero Segundo
- **Enlaces:** Texto del enlace. (target="_blank" abre en nueva pestaña).
- **Imágenes:** . El atributo alt es **obligatorio** para la accesibilidad y si la imagen no carga.

B. CSS3: Selectores y Reglas

Sintaxis de una regla CSS: selector { propiedad: valor; propiedad: valor; }

1. Métodos de Inclusión:

- **Externo (Recomendado):** Archivo .css separado vinculado con <link> en el <head> del HTML.
- **Interno:** Dentro de una etiqueta <style> en el <head>.
- **En línea (Desaconsejado):** Directamente en el atributo style de una etiqueta HTML. <p style="color: red;">. Rompe la separación de preocupaciones.

2. Selectores (Cómo elegir qué estilizar):

- **Universal:** * { ... } (Selecciona todos los elementos. Se usa para resets).
- **Tipo/Etiqueta:** p { color: blue; } (Todos los párrafos).
- **Clase (.):** .boton-rojo { background-color: red; } (Elementos con class="boton-rojo"). Es reutilizable.
- **ID (#):** #cabecera-principal { height: 100px; } (Elemento único con id="cabecera-principal"). Específico y único por página.
- **Descendiente (espacio):** article p { ... } (Párrafos que estén *dentro* de un article, en cualquier nivel).
- **Hijo directo (>):** nav > ul { ... } (ul que sea hijo directo de nav, no nieto).
- **Pseudo-clases (:):** Estados especiales. a:hover { ... } (cuando el ratón pasa sobre un enlace), input:focus { ... } (cuando el cursor está en un campo).

3. El Modelo de Caja (Box Model) - CONCEPTO FUNDAMENTAL:

Todo elemento en HTML es, en esencia, una caja rectangular. CSS controla las capas de esa

caja. Es vital entender esto para maquetar.

De adentro hacia afuera:

1. **Content (Contenido):** El texto, imagen o elementos hijos reales. Su tamaño se define con width y height.
2. **Padding (Relleno):** Espacio transparente *entre* el contenido y el borde. Aumenta el tamaño total visible de la caja.
3. **Border (Borde):** La línea que rodea el padding y el contenido. Tiene grosor, estilo y color.
4. **Margin (Margen):** Espacio transparente *fuera* del borde. Separa la caja de otros elementos vecinos. Los márgenes verticales a veces se "colapsan" (se fusionan).

El truco del box-sizing: border-box;

Por defecto, si defines width: 100px, padding: 20px y border: 5px, el ancho total real de la caja será $100 + 20 + 20 + 5 + 5 = 150\text{px}$. Esto hace que los cálculos de maquetación sean una pesadilla.

La "buena práctica" universal moderna es aplicar esto al inicio del CSS:

CSS

```
* {  
  box-sizing: border-box;  
}
```

Con esto, si defines width: 100px, el navegador ajustará el contenido para que el ancho total final (incluyendo padding y borde) sea exactamente 100px. Mucho más intuitivo.

4. Maquetación Moderna con Flexbox (Flexible Box Layout):

Es un modelo de diseño CSS3 diseñado para distribuir espacio y alinear elementos dentro de un contenedor, incluso cuando sus tamaños son desconocidos o dinámicos. Es unidimensional (maneja filas o columnas).

Conceptos clave:

- **Contenedor Flex:** Se activa con display: flex; en el elemento padre.
- **Ejes:** Eje Principal (por defecto horizontal, fila) y Eje Cruzado (vertical). Se pueden cambiar con flex-direction: column;

Propiedades principales del contenedor:

- **justify-content (Alineación en Eje Principal):** flex-start (inicio), flex-end (final), center (centro), space-between (separados a los extremos), space-around (espacio alrededor).

- **align-items (Alineación en Eje Cruzado):** stretch (estirar, por defecto), flex-start (arriba), flex-end (abajo), center (centrado vertical).
 - **flex-wrap:** nowrap (todo en una línea, por defecto), wrap (permite saltar a múltiples líneas si no cabe).
-

4.3. Ejemplo Práctico Completo: Página de Producto (Ficha de Coche)

Vamos a crear una página web completa para mostrar la ficha de un coche, utilizando HTML5 semántico y CSS3 moderno con Flexbox y el modelo de caja, integrando todo lo aprendido.

Archivo HTML: ficha_coche.html

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ford Focus RS - Concesionario Web</title>
  <link rel="stylesheet" href="estilos_coche.css">
</head>
<body>

  <header class="site-header">
    <div class="container header-content">
      <h1>Concesionario MotorSport</h1>
      <nav>
        <ul>
          <li><a href="#">Inicio</a></li>
          <li><a href="#" class="activo">Vehículos</a></li>
          <li><a href="#">Contacto</a></li>
        </ul>
      </nav>
```

```
</div>
</header>

<main class="container">
  <article class="ficha-producto">

    <div class="producto-imagen">
      
    </div>

    <div class="producto-detalles">
      <header>
        <p class="marca">Ford</p>
        <h2>Focus RS 2.3 EcoBoost</h2>
      </header>

      <section class="caracteristicas">
        <h3>Especificaciones</h3>
        <ul>
          <li><strong>Motor:</strong> Gasolina 350CV</li>
          <li><strong>Color:</strong> Blanco Nieve</li>
          <li><strong>Kilómetros:</strong> 45,000 km</li>
          <li><strong>Matrícula:</strong> 1234 BBC</li>
        </ul>
      </section>

      <section class="descripcion">
        <h3>Descripción</h3>
        <p>Vehículo deportivo de alto rendimiento en excelente estado. Tracción total, modos de conducción configurables y un motor explosivo. Ideal para entusiastas.</p>
      </section>

      <section class="precio-accion">
        <div class="precios">
          <p class="precio-anterior">35,000€</p>
          <p class="precio-actual">32,500€</p>
        </div>
        <button class="btn-comprar">Contactar Vendedor</button>
      </section>
    </div>
  </article>
</main>
```

```
<footer class="site-footer">
  <div class="container">
    <p>© 2023 Concesionario MotorSport. Todos los derechos reservados.</p>
  </div>
</footer>

</body>
</html>
```

Archivo CSS: estilos_coche.css

CSS

```
/* --- RESET BÁSICO Y VARIABLES --- */
:root {
  --color-primario: #004e92;
  --color-secundario: #f5f5f5;
  --color-texto: #333;
  --color-acento: #d32f2f;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* Crucial para el modelo de caja */
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: var(--color-secundario);
  color: var(--color-texto);
  line-height: 1.6;
}

/* --- CLASES UTILITARIAS --- */
.container {
  max-width: 1100px;
```

```
margin: 0 auto; /* Centrar horizontalmente */
padding: 0 20px;
}
```

```
/* --- HEADER GLOBAL --- */
```

```
.site-header {
  background-color: var(--color-primario);
  color: white;
  padding: 1rem 0;
  box-shadow: 0 2px 10px rgba(0,0,0,0.2);
}
```

```
/* Usamos Flexbox para alinear logo y menú */
```

```
.header-content {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
.site-header h1 {
  font-size: 1.5rem;
}
```

```
.site-header nav ul {
  list-style: none;
  display: flex; /* Menú horizontal */
  gap: 20px; /* Espacio entre enlaces */
}
```

```
.site-header nav a {
  text-decoration: none;
  color: rgba(255,255,255,0.8);
  font-weight: 500;
  transition: color 0.3s;
}
```

```
.site-header nav a:hover,
.site-header nav a.activo {
  color: white;
}
```

```
/* --- FICHA DE PRODUCTO PRINCIPAL --- */
```

```
main {
```

```
padding: 40px 0;  
}
```

```
.ficha-producto {  
  background-color: white;  
  border-radius: 10px;  
  overflow: hidden; /* Para que la imagen no se salga de las esquinas redondeadas */  
  box-shadow: 0 5px 15px rgba(0,0,0,0.1);  
  /* Flexbox para maquetación en dos columnas */  
  display: flex;  
}
```

```
/* Columna Izquierda: Imagen */
```

```
.producto-imagen {  
  flex: 1; /* Ocupa 1 parte del espacio disponible */  
  min-height: 400px;  
}
```

```
.producto-imagen img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover; /* La imagen cubre todo el espacio sin deformarse */  
}
```

```
/* Columna Derecha: Detalles */
```

```
.producto-detalles {  
  flex: 1; /* Ocupa 1 parte (50% / 50% con la imagen) */  
  padding: 30px;  
  display: flex;  
  flex-direction: column; /* Apilar secciones verticalmente */  
  gap: 25px;  
}
```

```
.producto-detalles header .marca {  
  text-transform: uppercase;  
  letter-spacing: 1px;  
  font-weight: 600;  
  color: #777;  
  font-size: 0.9rem;  
}
```

```
.producto-detalles h2 {  
  font-size: 2.2rem;
```

```
color: var(--color-primario);  
margin-top: 5px;  
}
```

```
.producto-detalles h3 {  
  font-size: 1.2rem;  
  margin-bottom: 15px;  
  border-bottom: 2px solid var(--color-secundario);  
  padding-bottom: 5px;  
}
```

```
.caracteristicas ul {  
  list-style: none;  
  display: grid;  
  grid-template-columns: 1fr 1fr; /* Grid de 2 columnas para características */  
  gap: 10px;  
}
```

```
.caracteristicas li {  
  background-color: var(--color-secundario);  
  padding: 8px 12px;  
  border-radius: 5px;  
  font-size: 0.95rem;  
}
```

```
.precio-accion {  
  margin-top: auto; /* Empuja esta sección al final del contenedor flex */  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding-top: 20px;  
  border-top: 1px solid #eee;  
}
```

```
.precio-anterior {  
  text-decoration: line-through;  
  color: #999;  
  font-size: 1.1rem;  
}
```

```
.precio-actual {  
  font-size: 2rem;  
  font-weight: bold;
```

```
    color: var(--color-acento);  
}
```

```
.btn-comprar {  
    background-color: var(--color-primario);  
    color: white;  
    border: none;  
    padding: 12px 25px;  
    font-size: 1rem;  
    font-weight: bold;  
    border-radius: 5px;  
    cursor: pointer;  
    transition: background-color 0.2s;  
}
```

```
.btn-comprar:hover {  
    background-color: #003366;  
}
```

```
/* --- FOOTER --- */  
.site-footer {  
    background-color: #333;  
    color: #aaa;  
    text-align: center;  
    padding: 20px 0;  
    margin-top: auto;  
}
```

```
/* --- MEDIA QUERY PARA DISEÑO RESPONSIVO (Móviles) --- */  
@media (max-width: 768px) {  
    .ficha-producto {  
        flex-direction: column; /* Cambiar a una sola columna vertical */  
    }
```

```
.producto-imagen {  
    min-height: 250px;  
}
```

```
.producto-detalles h2 {  
    font-size: 1.8rem;  
}
```

```
.precio-accion {
```

```
flex-direction: column;
gap: 20px;
}

.btn-comprar {
width: 100%;
}
}
```

Este ejemplo práctico final integra HTML5 semántico (header, nav, main, article, section, footer) con CSS3 avanzado usando variables, reset de modelo de caja, Flexbox para la estructura principal (cabecera y columnas del producto), Grid para detalles pequeños, y Media Queries para que se adapte perfectamente a dispositivos móviles.